Machine Learning for Named Entity Recognition and Classification Report

Melina Paxinou

Abstract

This project focuses on evaluating different machine learning models for Named Entity Recognition (NER) using the CoNLL 2003 shared task annotated data. The implemented models include Logistic Regression, Naive Bayes, Support Vector Machine (SVM), hyperparameter tuned SVM, and SVM using word embeddings combined with traditional one-hot features. In order to train and test each model, metrics such as precision, recall, and F1-score were used. Key findings include that the hyperparameter tuned SVM provides better generalization than other models, while the SVM using word embeddings combined with traditional one-hot features also delivered encouraging results.

1 Introduction

Named Entity Recognition (NER) is a core task in NLP that involves finding entities such as persons, locations, and organizations in text. This report shows the results of various machine learning models on the CoNLL 2003 shared task annotated data. Logistic Regression, Naive Bayes and Support Vector Machines (SVM) were tested with a focus on hyperparameter tuning for SVM and combination of word embeddings and one-hot encoded features. The aim was to compare the models and see how different feature sets and model configurations affect NER precision, recall, and F1-score metrics. The results show that SVM models, either with only traditional features or combined with word embeddings, are the most effective, and combining multiple feature types helps in improving classification quality.

2 Task and Data

2.1 Task

Named Entity Recognition and Classification (NERC) is used to identify named entities present within a text and assign them to specific categories, such as people, organizations, locations, and miscellaneous entities, as well as numeric expressions including time, date, money, and percent expressions. The purpose of NERC is to successfully detect named entities automatically in a text.

The CoNLL-2003 shared task (Tjong Kim Sang and De Meulder, 2003) focuses specifically on detecting the following labels: person, location, organization, and miscellaneous entity. The gold labels also contain BIO tags, B for the beginning of an entity, I for the inside of an entity, and O for non-entity tokens.

This specific project uses the annotated train, development, and test data provided by the CoNLL-2003 task for the creation of an NER system. This system invokes different classifiers, such as Logistic Regression and LinearSVC, different one-hot features, and the possibility to be combined with word embeddings.

2.2 Dataset and Distribution

A CoNLL file format represents each word on a separate line, including details such as the word, its syntactic category, its part-of-speech tag, and its NER label (e.g., B-PER or O). Sentences are separated by empty lines, clearly marking sentence boundaries.

Looking at the distribution of the data, I decided to examine the labels and the BIO tags together. In the training set, the most dominant label is B-LOC, followed closely by B-PER and B-ORG, and the least dominant label is I-MISC, although its number is very close to I-LOC. The development data distribution is similar to the train data, with B-PER being the most dominant class, followed by B-LOC. I-LOC is the least represented class, followed by I-MISC. An imbalance in the dataset could lead to misleading metrics, as it impacts how classification models perform during cross-validation, which determines how each class is represented in the dataset.

From the analysis of the POS tag, capitalization patterns and token length, I concluded that some potential features could be parts of speech and capitalization patterns. However, while token length could add to the shape of the token when combined with capitalization patterns, I do not believe that it could be beneficial to all classifiers.

2.3 Preprocessing

Preprocessing steps such as removing stopwords, lowercasing tokens, and removing punctuation would not be useful in NER, as named entities can contain stopwords, are usually in specific forms of capitalization, and contain punctuation marks (e.g. acronyms, abbreviations). Lemmatization may be detrimental to NER because, while it reduces the vocabulary size and improves processing efficiency, it also alters named entities in a way that makes them indistinguishable from non-named entities (e.g., "West" being reduced to a common noun). However, I defined a function that creates new conll files that contain all of the features in different columns, with the last column being the gold label.

2.4 Evaluation Metrics

When looking at the BIO system on a token level, we can immediately understand that there is a risk of the model classifying one token of a multi-token named entity correctly, while either wrongly classifying another or completely disregarding it and classifying it as "O". This is why span evaluation can be proven to be helpful in NER error analysis, as it gives us a better understanding of more complex cases.

All in all, the evaluation process should primarily focus on whether the classifier identified all named entities, as the worst outcome would be to disregard named entities as "O". If classifying both BIO and NER labels correctly is the best outcome, then simply classifying the position or label wrong would be a relatively acceptable result.

I am assessing the model on a token level, as it ensures quicker iterations during the development process, especially when time and processing power are limited.

To evaluate a classifier, precision, recall, and F1-score are the most commonly used metrics. Precision refers to how accurately the model identifies named entities, recall assesses its ability to find all relevant entities, and f1-score balances both to evaluate overall performance.

3 Models and Features

3.1 Models

Logistic Regression

Logistic regression is used in NER to classify a word or token to a specific named entity category. The model predicts the probability of each word having a specific NER label by examining features that we have given to it, such as part of speech tagging or capitalization patterns. It does so by estimating the probability of whether an instance belongs to a specific category or not. It is mostly used in binary classification tasks, where the possible classification outputs are in a positive/negative or True/False form; however, other types of Logistic Regression, such as Multinomial, can have more than two classes (Kanade, 2024).

Alternative Methods (SVM, NB, BERT)

The Naive Bayes classifier incorporates the MultinomialNB model from scikit-learn. It uses probabilities to predict classes by assuming that the presence of one word does not depend on the others. In this classifier, all features are independent of each other given the class label. Because of this, it is time-efficient and suitable for large datasets. MultinomialNB is specifically used for text data and uses probabilistic methods to predict the category of a text document based on the frequencies of words (Kavlakoglu, 2024).

SVM is used to map the data to a high-dimensional feature space. In this way, data points can be categorized, even when the data is not linearly separable. The algorithm finds the hyperplane that separates the points of one class from those of another class. By finding the hyperplane with the largest margin, it can be used to make predictions on new data (IBM, 2024). To make a prediction, the new data point is transformed into the same feature space as the training data, and then the class label is determined based on where it falls in that feature space (Fagbuyiro, 2024).

Bidirectional Encoder Representations from Transformers (BERT) is an NLP model that processes language bidirectionally, taking into account both the left and the right context of each word in a sentence. This approach allows BERT to understand the dependencies, discern nuances, and recognize relationships between words, thus analyzing at a depth that other types of model cannot achieve (Kocaman, 2024).

3.2 Features

Feature exploration

The tokens themselves are the first feature used in the first Logistic Regression classifier, as they contain several common words that are not named entities by default, such as articles, verbs, and prepositions. Therefore, they automatically help to exclude this common set of words from the NER recognition. The features which I chose to explore next were the part of speech tags of the original conll files and the capitalization patterns of each token. The part of speech tagging feature would help get a clear insight on whether the named entities follow some kind of pattern, such as commonly being proper nouns or adjectives. As for the capitalization patterns, capitalizing the first letter of people's names, or checking whether an acronym could be an organization, or identifying miscellaneous entities' names that often start with a capital letter, can be quite beneficial in NER. For capitalization patterns, I used four possible labels: uppercase, lowercase, first letter capitalization, and other, because I believe that these four types of capitalization patterns could prove useful with the cases mentioned and differentiate between the different types of named entity.

After developing my first classifier and seeing its results, I added three more features: syntactic role, previous tokens, and next tokens. The syntactic role could be important for the task, as identifying patterns between syntactic relations and named entities could be helpful to the classifier. For example, a named entity cannot be inside a verb phrase, but can be inside a noun phrase. Contextual representation is also important, as oftentimes named entities can be surrounded by the same prepositions (e.g. to, for) or can be found at the beginning of a sentence (in which case they would have the label "START!!" as their previous token). Specifically, the previous tokens are extremely important in NER classification, as apart from the prepositions or position in the sentence that were mentioned earlier, they can also be the beginning of a named entity, e.g. a first and last name, or the first part of a location. Next tokens may also be helpful, as there might be a pattern on what follows named entities (e.g. verbs), or they could provide the classifier with the second part of a two-word named entity.

Word embeddings as features add to the semantic aspect of each token. They act as numerical IDs of words, turning their specific nuances into numerical vectors and giving the model the opportunity to learn meaningful relationships and patterns. They facilitate NLP tasks such as text classification and sentiment analysis and enable models to understand and generate human-like text. They are created by providing a model with a large amount of text data and forming vector representations based on the context in which the words appear. In NER, word embeddings serve as features that can lead to correct classifications by allowing the model to comprehend the context and relationships between words (Olamendy, 2024).

4 Experiments and Results

4.1 Evaluation

The macro-average precision, recall, and F1-score of all the classifiers created in this project can be found in Table 1 below. The first logistic regression classifier that I created with only the token as a feature had a macro-average precision of 0.811, a recall of 0.547, and an F1-score of 0.637, which is a great starting point, considering that only one feature was available to it.

By adding all features to the Logistic Regression classifier, we can immediately see that the results have improved. Macro-average precision has increased by 0.066, recall by 0.256, and F1-score by 0.198. The results indicate that the added features are helpful for a logistic regression classifier.

In this step, two additional classifiers were added; MultinomialNB and LinearSVC. All the features mentioned were used directly on them, and the scores obtained are represented in Table 1.

MultinomialNB has substantial precision with a value of 0.846, but recall is much lower, with a value of 0.661. This stems from particularly low results in the I-LOC and I-MISC labels.

LinearSVC presented with the best results out of all three classifiers, which were consistently high for both precision (0.893) and recall (0.844), and resulted in a particularly high F1-score of 0.866.

Next, LinearSVC was combined with Google word embeddings and, as shown in Table 1, the classifier that only used the word embeddings did not get particularly high scores. Specifically, precision was 0.733, recall was 0.652, and F1-score was 0.686.

To combine traditional hot features with word embeddings, I used the word embeddings of the token, the previous token, and the next token, and

combined them with capitalization, syntax, and pos tag as one-hot features. When these features were combined, the results improved drastically. The precision climbed to 0.878, the recall was 0.836, and the F1 score was 0.855.

Lastly, three different seeds were tested with BERT to see the impact of the different seed values on precision, recall, and F1-score. The first seed value was 13, which resulted in the highest F1-score of 0.941. The second seed value was 127 and the third seed was 4321. They resulted in F1-scores of 0.937 and 0.94 respectively. All scores were high; however, recall was slightly higher across all the seed values (see Images 4, 5, and 6).

For the final experiment, the test data were used on the hyperparameter-tuned LinearSVC classifier. As shown in Table 2, the macro-average F1-score of the test data is significantly lower than that of the development data, dropping from 0.866 to 0.8. The overall precision has dropped more than the overall recall. The values of all classes are lower, yet I-MISC seems to have the most significant drop in precision. Despite this, the model maintains its performance in I-PER (0.878) and O (0.989), reinforcing its reliability to identify frequently occurring classes. In general, the model shows promising results, but struggles with generalizing on the rarer or more ambiguous classes, such as MISC.

Table 1: Performance Metrics for All Classifiers

Classifier	Precision	Recall	F1 Score
Logistic Regression	0.811	0.547	0.637
Logistic Regression (with all features)	0.877	0.803	0.835
Naive Bayes	0.846	0.661	0.699
Linear SVC	0.893	0.844	0.866
Embeddings SVC	0.733	0.652	0.686
Embeddings SVC (with all features)	0.878	0.836	0.855
Tuned Linear SVC	0.893	0.844	0.866

4.2 Hyperparameter Tuning

For hyper-parameter tuning, I focused on optimizing the C parameter of my LinearSVC. As time and processing power did not allow me to use a regular SVC, I opted to proceed with the LinearSVC I already had in place. I used the GridSearchCV method provided by scikit-learn to find the ideal C value, as a grid search systematically evaluates a range of potential C values to find the optimal one for my task (Scikit-learn, 2024a). Specifically, I performed an exhaustive grid search over the values [0.01, 0.1, 1, 10, 100] using 5-fold cross-validation. I also tested the loss parameter with values of ["hinge", "squared_hinge"] to test how each loss type affects the performance on the data. Lastly, I tested the max_iter parameter over the values [2000, 5000, 1000] to ensure convergence (Scikit-learn, 2024c).

For scoring, I selected the f1 macro metric. As there is often class imbalance in NER, the F1 macro score would be the ideal metric to measure each class independently and average them equally, as the O class is overrepresented and would overshadow the rest of the results.

I used cross-validation during tuning instead of development data splitting, as my goal was to reduce the risk of overfitting. Cross-validation would ensure the generalization performance of LinearSVC by evaluating it across multiple folds.

In retrospect, the results were not as promising as expected. In fact, they were exactly the same as the original results, as the hyperparameter-tuned LinearSVC results were the default values. The only parameter that was changed was max_iter with a value of 2000.

For the same process with a regular SVC, I would like to focus on testing C and gamma values. Specifically, the C value is the most relevant to investigate, as it defines the flexibility of the model and the penalization of misclassifications. Next, I would test different gamma values, such as [1, 0.1, 0.01, 0.001], as it defines the influence of a single training example. Non-linear kernels could also be considered for SVM with word embeddings.

4.3 Feature Ablation

I ran a full feature ablation script for my Logistic Regression classifier, as it was the first classifier used in this assignment and the basis for my feature development.

Table 3 describes the contribution of each characteristic when combined with the token itself, i.e., the original feature of this classifier. As expected, the token as a feature worked well enough on its own, with an F1-score of 0.638. The most impressive combination was the token and the previous token, with an F1-score of 0.762, followed by the next token and the capitalization pattern. The POS tag and syntax did not contribute as much, but also did not worsen the results of the classifier.

Moreover, Table 4 presents the results of the fea-

Class	Precision (Dev)	Recall (Dev)	F1-Score (Dev)	Precision (Test)	Recall (Test)	F1-Score (Test)
B-LOC	0.911	0.868	0.889	0.858	0.824	0.841
B-MISC	0.895	0.822	0.857	0.789	0.769	0.779
B-ORG	0.847	0.808	0.827	0.786	0.734	0.759
B-PER	0.900	0.915	0.908	0.810	0.840	0.825
I-LOC	0.893	0.809	0.849	0.734	0.708	0.721
I-MISC	0.848	0.679	0.754	0.654	0.657	0.656
I-ORG	0.858	0.738	0.793	0.807	0.704	0.752
I-PER	0.895	0.956	0.925	0.816	0.950	0.878
O	0.990	0.997	0.994	0.988	0.989	0.989
Macro Avg	0.893	0.844	0.866	0.805	0.797	0.800

Table 2: Comparison of NER results between development and test datasets using the LinearSVC classifier.

ture ablation in which one feature is removed from the full set of features. The classifier indeed works best when all features are added to it; however, the token itself and the previous token have the most impact on the classifier when removed. Next token and capitalization also seem to improve it and POS tag and syntax are the least useful features.

Feature Combination	F1-Score
Token	0.638
Token + POS	0.687
Token + Syntax	0.676
Token + Capitalization	0.691
Token + Prev_Token	0.762
Token + Next_Token	0.702

Table 3: Impact of Token and Two-Feature Combinations on F-Score for Logistic Regression

Removed feature	F1-Score
Next_token	0.8105
Prev_token	0.7865
Capitalization	0.8299
Syntax	0.8353
POS	0.8322
Token	0.7301
Total score	0.8378

Table 4: Impact of Multi-Feature Combinations with One Feature Removed on F-Score for Logistic Regression

In Table 5, the combinations with the highest F1-scores (above 0.8) are tested in the Naive Bayes and the LinearSVC classifiers. The combination ['token', 'capitalization', 'prev token', 'next_token'] was the highest scoring combination on the NB classifier, with an F1-score of 0.728, and the combination that included all features was the best for the hyper-parameter tuned LinearSVC with an F1 score of 0.865.

All in all, a consistently strong combination of features is the token itself, the previous token, and the capitalization pattern. The other features mostly add value to the Logistic Regression and LinearSVC classifiers. This possibly stems from

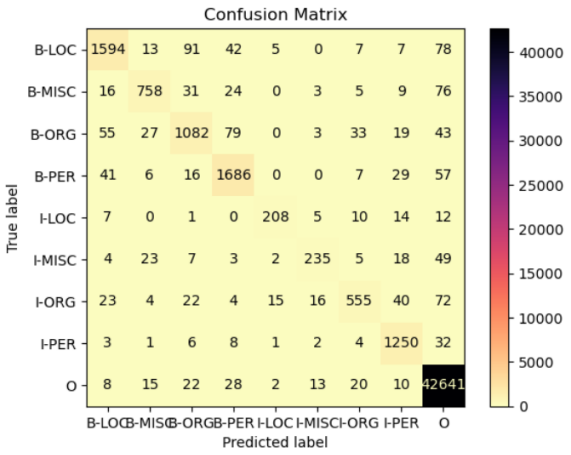

Figure 1: Hyperparameter tuned SVM Confusion Matrix

Naive Bayes' perception of features with conditional independence.

5 Error Analysis

As shown in Figure 1, LinearSVC performs well for B-PER, with 1866 correct predictions, although there are some misclassifications, with 41 false positives (as B-LOC) and 57 false negatives (as O). Similarly, B-LOC and B-ORG show high accuracy, with the most significant misclassifications being in other categories such as I-LOC and I-ORG. The O label is correctly classified in most cases (42641) and the errors are insignificant compared to the number of entities in the O class. I-MISC and I-PER show a moderate number of misclassifications but still perform reasonably well, with a mix of confusion across the categories.

Deeper into error analysis, the number of gold, predicted, and correctly predicted labels of lowercase named entities was analyzed. In total, out of the 122 entities in the development dataset, the correctly classified entities were 60 and the incorrectly classified were 62. In Table 6, we can see that

Feature Combination	LogReg F1 Score	NB F1 Score	SVC F1 Score
Token + POS + Syntax + Capitalization + Prev_Token + Next_Token	**0.8377**	0.6907	**0.8668**
Token + Syntax + Capitalization + Prev_Token + Next_Token	0.8322	0.7048	0.8663
Token + POS + Capitalization + Prev_Token + Next_Token	0.8352	0.6923	0.8660
Token + POS + Syntax + Prev_Token + Next_Token	0.8298	0.6986	0.8592
Token + POS + Syntax + Capitalization + Prev_Token	0.8105	0.6881	0.8381
Token + Syntax + Prev_Token + Next_Token	0.8	0.6762	0.8471
Token + Syntax + Capitalization + Prev_Token	0.8096	0.6978	0.8372
Token + POS + Capitalization + Prev_Token	0.8272	0.6851	0.8366
Token + Capitalizaton + Prev_Token + Next_Token	0.834	**0.7318**	0.8643
Token + Capitalization + Prev_Token	0.8056	0.7287	0.8348

Table 5: F1 Scores for Best Feature Combinations for Naive Bayes (NB) and Linear SVC Classifiers according to the feature ablation results of LogReg

Label	Counts		
	Gold	Predicted	Correct
I-ORG	62	31	31
I-LOC	10	4	4
I-PER	20	19	17
I-MISC	24	6	5
B-MISC	4	1	1
B-PER	1	1	1
B-ORG	1	1	1
O	0	59	0

Table 6: Label Statistics for Lowercase Entities

59 lowercase entities were misclassified as O. The gold labels of these entities were mostly I-ORG and I-MISC, and most of the tokens were either "of", "the", "and", "in", and "'s". These misclassifications could be attributed to the presence of stop words, such as "of" and "the", which do not clearly indicate an entity. Figure 3 describes the actual distribution of labels among the lowercase named entities, all predicted labels for these entities, and correctly predicted labels for these entities.

It is important to note that all "der" lowercase named entities were classified correctly, which most likely stems from the fact that they are not English words, and therefore the system managed to identify them as entities. The "van" entities were also classified correctly, which is impressive, considering that there is an English word "van" that the model could have confused it with.

To improve the model in this area, span evaluation could be considered. As these tokens are mostly in the I category, they are smaller parts of larger named entities, such as location names (e.g. Federal Republic of Yugoslavia), miscellaneous entities (e.g. Duke of Norfolk), and organization names (e.g. Boatmen's). A span-based approach would treat the tokens of a multi-token named entity as one and may have more chances in classifying them correctly.

Moreover, bidirectional models, such as BERT, would most likely not have this issue. As most of these "stop word" tokens are usually inside a named entity surrounded by other tokens (e.g. United States of America), such models would present with fewer issues in identifying them.

6 Discussion

Despite achieving promising results, several common challenges in NER were encountered during this project. Although feature engineering was beneficial to all classifiers, there is room for improvement in handling features POS tags and syntactic relations, for example, by using Boolean values for keys such as NNP and JJ, the most common tags found in the train and development sets. Span evaluation could also be considered for more accurate results on the entirety of named entities, particularly for partial or overlapping spans. Addressing class imbalance in the dataset could also help improve performance, as certain classes were underrepresented. Time and resources allowing, experimentation on an SVC, instead of LinearSVC, could also be proven insightful when handling non-linear decision boundaries, such as word embeddings.

7 Conclusion

This project demonstrated the effectiveness of feature engineering and machine learning models in Named Entity Recognition tasks. Although the results were promising when working with classifiers such as LinearSVC and Logistic Regression, addressing feature limitations and class imbalance has room for improvement. Future exploration of advanced models, such as kernel-based SVCs, could help overcome some of these challenges. In general, the findings highlight the importance of feature engineering and model selection in developing robust NER systems.

Acknowledgements

I would like to acknowledge Elisabetta Dentico for her collaboration in analyzing the dataset statistics, selecting features for our classifiers, and developing the three different classifiers. I also worked with Martha Koukourikou to determine the best options for hyperparameter tuning. Additionally, the feature ablation code was inspired by Nikhil Matthews (Matt).

References

Sumit Dua. 2024. Text classification using k-nearest neighbors. Retrieved on December 15, 2024.

D. Fagbuyiro. 2024. Understanding the support vector machine (svm) model. Retrieved on December 15, 2024.

GeeksforGeeks. 2024. K-nearest neighbours. Retrieved on December 15, 2024.

IBM. 2024. How svm works. Retrieved on December 15, 2024.

V. Kanade. 2024. What is logistic regression? Retrieved on December 15, 2024.

E. Kavlakoglu. 2024. Classifying data with the multinomial naive bayes algorithm. Accessed: December 15, 2024.

A. M. Kocaman. 2024. Mastering named entity recognition with bert: A comprehensive guide. Accessed: December 15, 2024.

Heeyoung Lee, Angel Chang, Yves Peirsman, Nathanael Chambers, Mihai Surdeanu, and Dan Jurafsky. 2013. Deterministic coreference resolution based on entity-centric, precision-ranked rules. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 865–876. Association for Computational Linguistics.

Preslav Nakov, Zornitsa Kozareva, Alan Ritter, Sara Rosenthal, Veselin Stoyanov, and Theresa Wilson. 2013. Semeval-2013 task 2: Sentiment analysis in twitter. In *Proceedings of the 7th International Workshop on Semantic Evaluation (SemEval 2013)*, pages 312–320. Association for Computational Linguistics.

J. C. Olamendy. 2024. Unlocking the power of text classification with embeddings. Retrieved on December 15, 2024.

ScienceDirect. 2024. K-nearest neighbor classifier. Retrieved on December 15, 2024.

Scikit-learn. 2024a. Scikit-learn grid search: Hyperparameter tuning for linearsvc. Retrieved on December 15, 2024.

Scikit-learn. 2024b. sklearn.neighbors.kneighborsclassifier. Retrieved on December 15, 2024.

Scikit-learn. 2024c. sklearn.svm.linearsvc. Retrieved on December 15, 2024.

E. F. Tjong Kim Sang and F. De Meulder. 2003. Introduction to the conll-2003 shared task: Language-independent named entity recognition. *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147. Retrieved on December 11, 2024.

Theoretical Component

Machine learning is when a computer learns from data instead of following fixed instructions. Specifically, a machine can learn by collecting and analyzing data and grouping information into categories. Its performance improves by using data without the need for explicit programming and instructions. This can be achieved in several ways — supervised, semi-supervised, unsupervised, or self-supervised learning — and it can continue to evolve as long as it receives new data.

Sentiment analysis

A1. Message polarity classification is a classification task that aims to identify positive or negative emotions in a message and label it as positive, negative, or neutral, depending on which sentiment is stronger (Nakov et al., 2013). The input of this task depends on whether the system is constrained or unconstrained; a constrained system can only use the provided data, as well as other resources such as lexicons, whereas an unconstrained system can use any additional data in a supervised, semi-supervised, or unsupervised learning approach.

A2. In sentiment classification, several features can be used: word-related, syntactic, domain-specific, or sentiment-related. Specifically, the word itself can be used, either in its original form or lemmatized. By lemmatizing a word, we treat all variants of the same word as one and reduce sparsity (e.g. have, has, had as have). Stop word removal can be an option; however, using a curated list of stop words that does not include negation is necessary. Capitalization can also be useful, as oftentimes using only or mostly uppercase letters can signify strong emotions compared to a sentence that only has lowercase letters. Punctuation works in a similar way, as excessive punctuation or certain types of punctuation can give us more insight on the specific sentiment (e.g. "I don't know" vs. "I DON'T KNOW!!!"). As for the syntactic relations, POS tags and dependency relations can help with disambiguation (e.g., "likes", which can either be a social media feature or a verb). Moreover, the context of a word can be proven useful, as it can contain negation (e.g. "not happy"), as well as content preprocessing and chunking (e.g. using bi-grams to catch negation patterns similarly to the previous example). Domain-specific features can also be crucial, for example in a tweet classification task, we would need to use emojis, Twitter-specific dictionaries (e.g. containing slang, abbreviations), and hashtag content, but we can also use general sentiment dictionaries, which contain emotional words with a specific weight attached to them, for example giving "amazing" more weight than "fine", even though both are positive words. However, sentence length is unlikely to indicate sentiments effectively, as oftentimes having very long sentences or very short sentences can carry the same meaning (e.g., "I hate you!!" vs. a sentence which explicitly states all the reasons why this person hates the other).

A3. The system sees all previously mentioned features as numerical representations and attempts to find patterns between features and sentiment categories. Each tweet is tokenized, and for each token, POS tags are extracted. POS tags are then converted into one-hot vectors that correspond to all possible tags in the training data. Hashtags are processed to extract their content, which is either lemmatized or directly encoded as one-hot vectors, depending on the context. A numerical vector, for example, can hold information on a word's lemma or context, which the system uses to find correlations. Humans can understand this information intuitively, by looking at this kind of information all at once. For example, when we read a tweet that contains several exclamation marks, angry emojis, and all uppercase letters, we immediately understand that the sentiment is negative.

Pre-trained word embeddings could be used to encode tokens in the tweet. Each token is represented as a 300-dimensional vector, and token embeddings could be combined with average pooling to produce a tweet-level embedding. This embedding is concatenated with other features such as capitalization and punctuation patterns to form the final input vector for classification.

Coreference resolution

B1. Coreference resolution is a task aimed at finding and linking all expressions that refer to the same named entity in a text (Lee et al., 2013). It uses pronouns, noun phrases, and proper nouns to link named entities to all their references in that text. For example, in the sentence "Stella is a programmer. She created a new function," the system would have to identify that "Stella", "She", and "a programmer" refer to the same named entity.

The system would take as input text with annotated mentions. Specifically, the text would have to be tokenized and annotated (e.g. with part-of-speech tags) and then assigned specific numbers, i.e. "[Stella] is [a programmer]. [She] created [a

new function].” Then, the system would ideally provide us with clusters (groups of mentions that refer to the same entity) as output. In the given example, “Stella”, “a programmer”, and “She” would be in one cluster.

B2. As this task requires accurate identification of both the entity mentions and the relationship between them, according to Lee et al., useful features would include information both on the named entity itself and on previous entity mentions already linked to a cluster. Firstly, using NER labels as features is crucial to coreference resolution, as knowing if there is a named entity present in the text is the first step in this task. If the system knows whether it has come across a named entity, in theory it could more easily identify the words or phrases that refer to this named entity.

Next, syntactic features could be proven useful, as identifying subject-verb-object relations may help link pronouns to referents. In the given example, “Stella” is the proper noun, and “She” is the pronoun that follows in the next sentence; therefore, the system would be able to identify the relationship based on the dependency relations.

To reinforce the link between such examples, features such as gender and number agreements could be proven helpful. Following the same example, “Stella” is a name commonly associated with people who identify as female, and “She” is a female pronoun. The singular form also eliminates referents in plural form and makes the link between the two even stronger. In a similar fashion, person attributes may also be a valuable feature; however, a distinction should be made when quotes are present in the text. For example, in *“I am a programmer,” Stella said*, “Stella” and “I” refer to the same named entity.

Semantic features, such as word embeddings, could be used to categorize based on similarity. For example, "car" and "vehicle" could refer to the same object. Similarly, definite and indefinite mentions could also be useful, as “a car” and “the car” do not have the same referent.

Other features could include animacy and pronoun distance. Specifically, animacy refers to whether a word or phrase refers to a living being, such as humans and animals, or to an inanimate object. In the given example, “Stella” is animate, but “a new function” is not. The distance between pronouns could also play a significant role, as a larger number of sentences separating two pronouns increases the likelihood that the latter pronoun does not share the same referent.

B3. To feed features into the system, we would have to represent them numerically. Several of the features mentioned above can be represented as one-hot vectors, such as NER labels, gender, number, and syntactic relations. For example, if we were to classify “Stella”, and considering that NER entities include person, location, organization, and miscellaneous entity, the one-hot vector representing this feature would look as follows: [1, 0, 0, 0]. While humans view this information intuitively and contextually, machines only use this kind of feature to narrow down the possibilities. We would automatically know that “Stella” and “She” refer to the same person; however, a computer would only have the information that “Stella” is a person and would not be categorized in a cluster containing a different type of entity.

The same can be used for all similar features. The gender could be transformed into [0, 1, 0] for masculine, feminine, and neutral, and the number could simply be binary, with zero for singular and one for plural. As a result, the machine could combine the information from all the vectors available to it and identify the cases where they all match.

However, as mentioned before in the example for the person feature, not everything is as clear. In the example where quotes are present, “Stella” and “I” refer to the same person, therefore a condition would need to be applied for some of the features to be optional.

On the other hand, word embeddings as features would be represented in a different way for a machine to understand. As they hold a large amount of semantic and contextual information, they would be seen by a machine as dense numerical vectors. They would help clarify similarities between words, as for example “car” would have a vector close to “vehicle”. Similarly to the features mentioned above, humans understand this kind of information automatically and at the same time, whereas a computer would need to process all those features and approximate the connections between them.

K-Nearest Neighbors Classifier

1. The K-Nearest Neighbors (KNN) algorithm is a machine learning algorithm used for classification. It is used in supervised learning and can be proven quite useful, as it does not make any assumptions about the data distribution (ScienceDirect, 2024). Its mechanism works by classifying the

coordinates into groups identified by an attribute. In the example of training instances, the green triangles are allocated in one group and the yellow squares in another. Then, the classifier would use the algorithm to categorize any new data based on the group to which its nearest neighbors belong (Dua, 2024). For example, a point close to a cluster of points classified as green triangles would be perceived as a green triangle.

The benefits of this type of classifier are the simplicity of its algorithm and its ease of implementation. It can handle both numerical and categorical data and can be adjusted to different types of datasets and tasks. It does not take parameters and works with the similarity of data points in a dataset.

The most common distance metrics include the Euclidian distance, Manhattan distance, and cosine similarity (Scikit-learn, 2024b). Euclidian distance refers to the length of the visualization of a straight line that unites the two points. Manhattan distance refers to the sum of the absolute difference between the coordinates in the n-dimensions. Lastly, cosine similarity is used to determine the optimal number of neighbors, with the data points that present the highest similarity being prioritized as the closest, and those with the lowest similarity are excluded (GeeksforGeeks, 2024).

2. If we look closely at Figure 2, we can form a hypothesis in which category the new instance will end up. Assuming that k=3, since the red circle only includes three data points and that the three data points included are its nearest neighbors, the new instance will be classified as a green triangle. If k=2, the classification outcome changes drastically. As two is an even number, and the two nearest neighbors of the new instance are members of different groups, the new instance would remain unclassified. However, if k=5, then the 5 nearest neighbors of the new instance would be three green triangles and two yellow squares; therefore, the new instance would be classified as a green triangle.

3. Assigning k with an appropriate value is essential as it plays an important role in the classification process. Specifically, assigning k with a very small number, such as 1, would mean that the classifier would only take into account one neighbor. In Figure 2, the new instance would be classified as a yellow square, even though most of its nearest neighbors are green triangles. Similarly, if k was a very large number, the search would become too

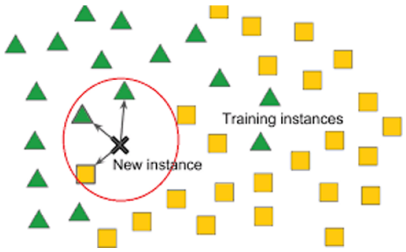

Figure 2: Example of training instances and instance to classify

wide and the results would be unreliable. If, for example, the nearest neighbors of this new instance were mostly green triangles when k=5 (that is, a relatively small number), but the density of yellow squares is greater, then increasing k to include more yellow squares would drastically change the results. A larger k could potentially help with the classification of instances that could belong to many different groups, as more groups could be included in the calculation; however, it would not be useful for classifications with fewer groups.

Time spent

Week	Task	Time
1	class hours	5h
1	videos for week 1	1h20
1	understand how latex works	1h
1	understand labels in data (code)	30min
1	write text for 1, 2.1	30min
1	code and planning of 2.2	4h
1	report and confusion matrix	1h
2	class hours	5h
2	videos for week 2	1h20
2	understanding of the model creation code	2h
2	writing sections 2.2 to 2.4	3h
2	code for model creation, additional features, report and confusion matrix	2h
2	writing sections 3.1 and 3.2	1h
3	class hours	5h
3	videos week 3	50min
3	theoretical part (A)	2h
3	code for week 3 (two more features, different models)	2h
3	evaluation: first results	1h
4	class hours	5h
4	embeddings and coursera videos	3h
4	embeddings code	3h
4	readings	2h
4	writing of models and features	1h
5	class hours	3h
5	readings	3h
5	embeddings code and understanding	8h
6	class hours	5h
6	transformers video	30min
6	coreference resolution	3h
6	KNN	4h
6	feature ablation	3h
6	running bert (not added to total time)	18h
6	hyper parameter tuning	3h
7	feedback doc implementation	40h
7	study for exam	30
Total	156h40min	

Table 7: Time overview.

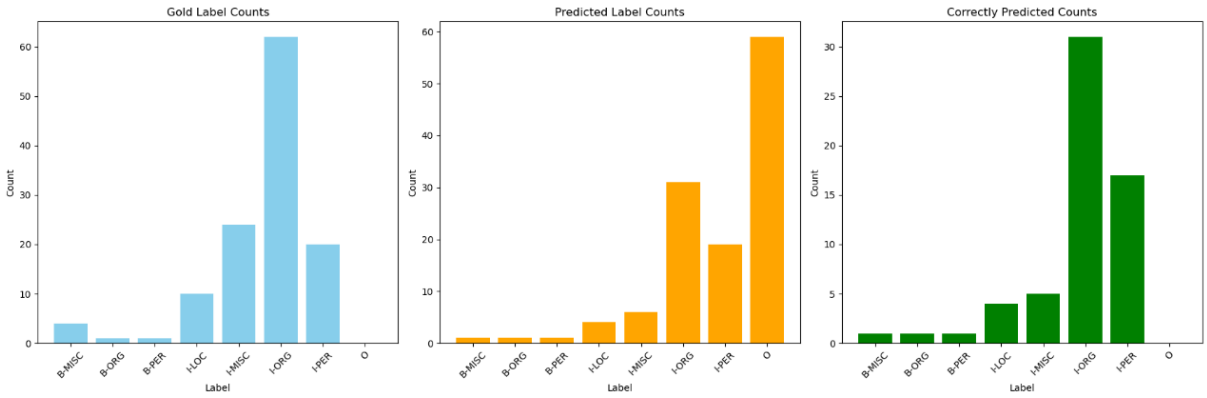

Figure 3: Error analysis for lowercase named entities - development data


```
{'LOC': {'precision': 0.9563417388031614,
  'recall': 0.9705882352941176,
  'f1': 0.9634123222748815,
  'number': 2618},
 'MISC': {'precision': 0.8261886204208886,
  'recall': 0.8610885458976442,
  'f1': 0.8432776451869531,
  'number': 1231},
 'ORG': {'precision': 0.9113984674329502,
  'recall': 0.9255836575875487,
  'f1': 0.9184362934362935,
  'number': 2056},
 'PER': {'precision': 0.9760577238438832,
  'recall': 0.980883322346737,
  'f1': 0.9784645734012822,
  'number': 3034},
 'overall_precision': 0.9342293709375344,
 'overall_recall': 0.9486519744937912,
 'overall_f1': 0.941385435168739,
 'overall_accuracy': 0.9860835305892258}
```


Figure 4: BERT: 13


```
{'LOC': {'precision': 0.9570459683496609,
  'recall': 0.9702062643239114,
  'f1': 0.9635811836115327,
  'number': 2618},
 'MISC': {'precision': 0.8270142180094787,
  'recall': 0.8505280259951259,
  'f1': 0.8386063275931117,
  'number': 1231},
 'ORG': {'precision': 0.9039655996177736,
  'recall': 0.9202334630350194,
  'f1': 0.9120269944564956,
  'number': 2056},
 'PER': {'precision': 0.9730351857941467,
  'recall': 0.9752801582069874,
  'f1': 0.974156378600823,
  'number': 3034},
 'overall_precision': 0.9319637729180472,
 'overall_recall': 0.9439534623559682,
 'overall_f1': 0.9379203023397988,
 'overall_accuracy': 0.9853050979395365}
```


Figure 5: BERT: 127

```
{'LOC': {'precision': 0.9508811398575178,
  'recall': 0.9686783804430863,
  'f1': 0.9596972563859981,
  'number': 2618},
 'MISC': {'precision': 0.839584996009577,
  'recall': 0.8545897644191714,
  'f1': 0.8470209339774557,
  'number': 1231},
 'ORG': {'precision': 0.9102256361017763,
  'recall': 0.9221789883268483,
  'f1': 0.9161633244745107,
  'number': 2056},
 'PER': {'precision': 0.9757377049180328,
  'recall': 0.980883322346737,
  'f1': 0.9783037475345169,
  'number': 3034},
 'overall_precision': 0.9344968518723076,
 'overall_recall': 0.9464145877614946,
 'overall_f1': 0.9404179635393509,
 'overall_accuracy': 0.9862741671564967}
```


Figure 6: BERT: 4321

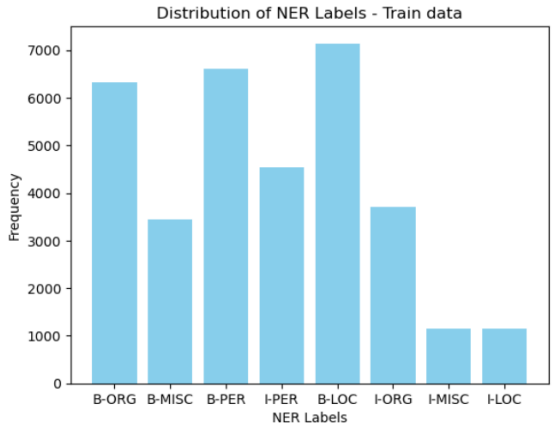

Figure 7: Distribution of NER labels - train data

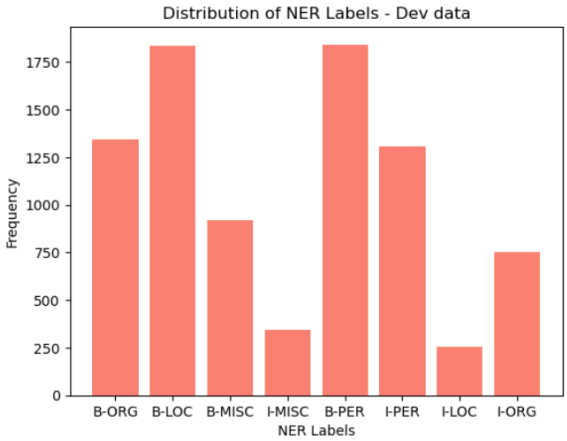

Figure 8: Distribution of NER labels - development data

NERC-research preparation

Dataset and Distribution

In order to understand the train and development data and perform feature engineering, analysis was performed on feature and label distributions. Label frequency, named entity frequency, POS tag distribution, capitalization patterns, and token length analysis were performed in both files.

As shown in Figure 7, the training set's dominant label is B-LOC, followed by B-PER and B-ORG. I-PER, I-ORG, B-MISC are next, with the least represented labels being I-LOC and I-MISC. In Figure 8, a similar pattern is followed. It is clear that in both sets the I-LOC and I-MISC classes are gravely underrepresented.

In the POS tag analysis, it was revealed that in both sets the most common POS tag was NNP by far. This stems from the fact that most named entities are proper nouns. However, the miscellaneous entity category contained several adjectives (JJ tag), which is explained by the fact that many of these

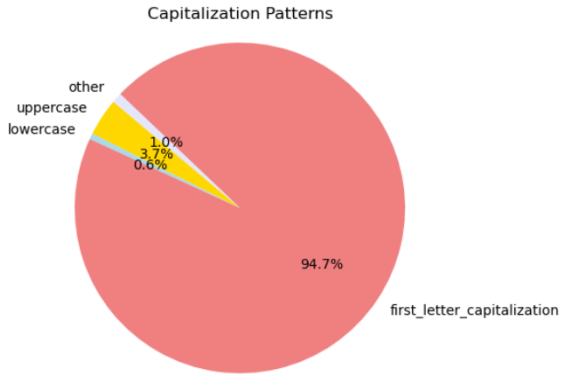

Figure 9: Capitalization patterns for Person - train data

entities are nationalities of people.

The capitalization pattern analysis was proven to be extremely useful for this task. Specifically, all named entities were placed into the following categories: uppercase, lowercase, first letter capitalization, and other. Named entities can have various forms of capitalization that may help identify their classes more easily. Figure 9 portrays how first letter capitalization dominates the Person entities in the train data, which is not unreasonable, considering that most Person entities are names. The Location entities in Figure 10 follow the same pattern, however in this case uppercase entities are much more than in the previous class. Organization entities follow the Location pattern, although an increase in the lowercase entities seems to appear, most likely due to the fact that some organizations may contain prepositions. Lastly, in the Miscellaneous entity class, there is greater variety. As shown in Figure 12, first letter capitalization dominates the chart, as in all other entity charts, with uppercase following, however the other category has a slightly bigger presence. Lastly, as shown in Figures 13, 14, 15, and 16, the development data has an almost identical distribution.

The final feature explored in this analysis is token length. The token length of each entity spans from 1 to 18 characters, however, as seen in Table 8, all categories show similar distribution. It may be proven useful when combined with other features, such as capitalization patterns, yet the interaction of features may not work well with some classifiers.

Evaluation Metrics

Precision, recall, and F1-score are metrics used to evaluate the performance of a model, especially in classification tasks such as Named Entity Recogni-

Table 8: Token Length Statistics for Named Entity Categories

Entity Type	Average Length	Max Length	Min Length
PERSON	6.05	18	1
LOCATION	6.33	17	1
ORGANIZATION	6.29	18	1
MISCELLANEOUS ENTITY	6.23	18	1

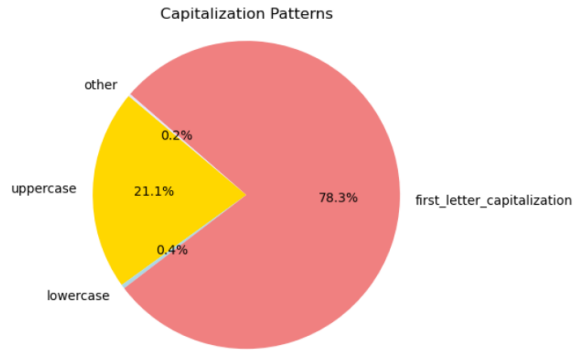

Figure 10: Capitalization patterns for Location - train data

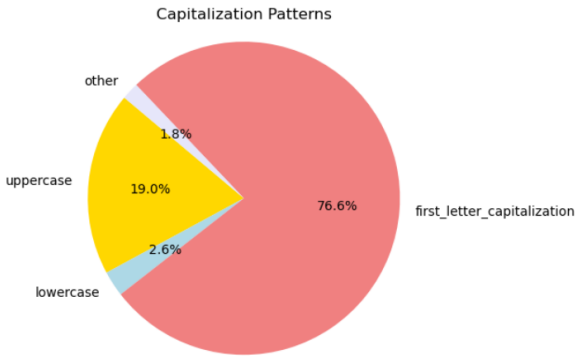

Figure 11: Capitalization patterns for Organization - train data

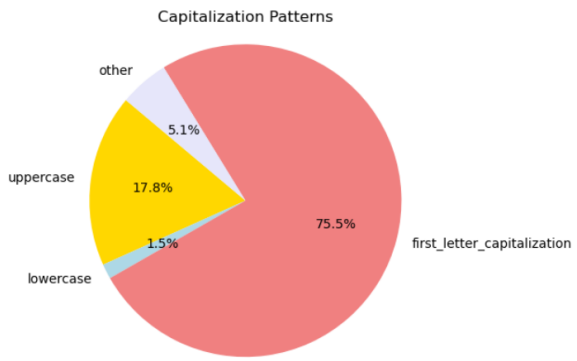

Figure 12: Capitalization patterns for Miscellaneous entity - train data

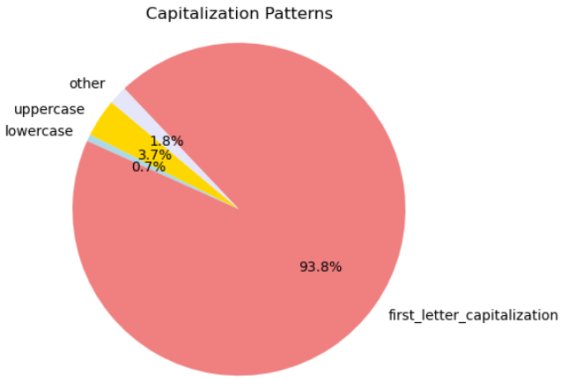

Figure 13: Capitalization patterns for Person entity - development data

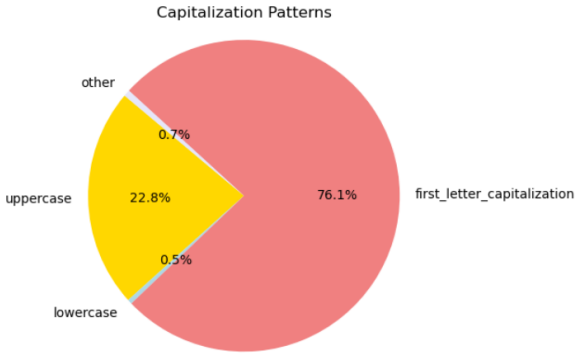

Figure 14: Capitalization patterns for Location - development data

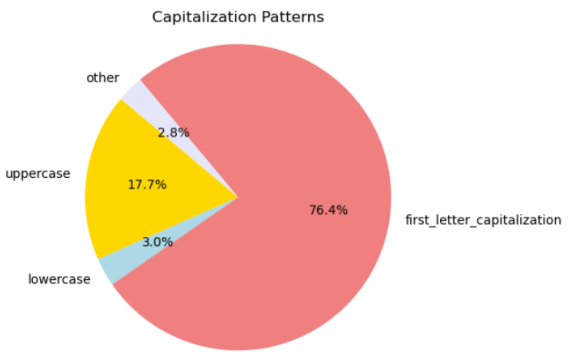

Figure 15: Capitalization patterns for Organization - development data

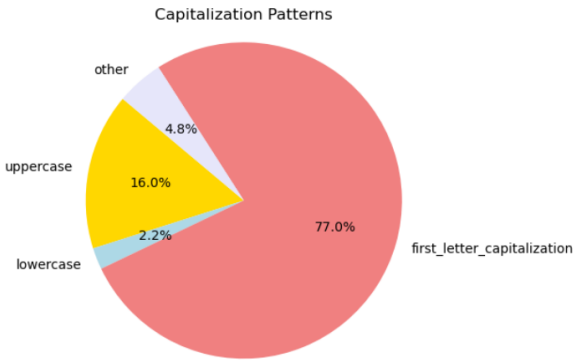

Figure 16: Capitalization patterns for Miscellaneous entity - development data

tion. Precision refers to how accurately the model identifies named entities, recall assesses its ability to find all relevant entities, and F1-score balances both to evaluate overall performance. High precision indicates that most predictions made by the model are correct, while high recall suggests that the model can detect most entities, even if it occasionally mislabels non-entities.

For example, in a medical NER system dedicated to the extraction of drug names along with patient records, high precision is crucial to prevent the issuing of incorrect prescriptions or misdiagnosis. In contrast, recall is crucial in a scenario where the model needs to identify as many cases as possible, such as in a news classification system. The model would need to locate as many entities as possible to classify the different news articles in different topics, even if in some cases it captures O elements as named entities.

Depending on the task at hand, a decision is needed on a token or a span evaluation. A token evaluation is useful to measure how well the model classifies individual tokens, whereas a span evaluation can capture named entities that consist of more than one token. Token evaluation is useful for the evaluation of the model's ability to find all tokens in a named entity, while a span evaluation is beneficial for understanding whether the model can identify the boundaries of a multi-token named entity.

However, evaluation at a token level may not fully capture the model's ability to recognize complete entities, especially when using the BIO system. For example, if the model identifies part of an entity, such as the *Beginning* token, but does not capture the *Inside* tokens, the entity is effectively misidentified, even though part of it was correctly labeled. Span evaluation is used to address this issue, as the system is judged on its ability to detect complete entities rather than individual tokens. This can be useful in cases where the correct identification of whole entities is critical. Combining token-level and span-level evaluations can provide a more comprehensive assessment of the model's performance.

Results

Logistic Regression

The first Logistic Regression classifier with only token as a feature did not perform very well. It had a relatively high precision, but very low recall, which then resulted in a low F1 score, as shown in Figure 17.

The results changed drastically when all the features were added, especially for the recall, which increased to 0.803 (Figure 18). As a result, the F1-score is much higher than the first version of this classifier. As for individual label performance, all labels have better prediction values; however, B-PER and I-ORG had the most impressive improvement in precision values, increasing to 0.864 and 0.825 respectively. The results for individual recall scores have also improved, especially for I-PER, which is now 0.937 with the new features.

Naive Bayes

The metrics for Naive Bayes are possibly the most inconsistent among all classifiers (Figure 19. Specifically, while precision is high (0.846), recall is much lower (0.661). This value is a result of the particularly low recall for I-LOC and I-MISC, with values of 0.198 and 0.231 respectively, whereas all the other categories have much better percentages. As in all other classifiers, the O category has almost excellent results in both precision and recall, and I-PER follows with 0.871 in both metrics. Almost all other categories are inconsistent; half of them have extremely high precision (e.g. I-MISC with a precision of 0.976 and a recall of 0.231), or extremely high recall (e.g. B-LOC with a precision of 0.666 and a recall of 0.911).

Note that during the feature exploration stage of this project, the "token length" was tested as a feature. While it seemed to be improving all other classifiers and contributing to the overall shape of each token, it resulted in some classes being 0 in the Naive Bayes classifier. This might be related to the conditional independence of features that characterizes Naive Bayes as a classifier. Therefore, the

Predicted Gold	B-LOC	B-MISC	B-ORG	B-PER	I-LOC	I-MISC	I-ORG	I-PER	O
B-LOC	1305	15	101	4	7	0	6	4	395
B-MISC	41	603	14	8	0	12	2	1	241
B-ORG	78	23	690	5	11	3	38	14	479
B-PER	16	3	2	873	0	0	1	104	843
I-LOC	13	2	1	0	150	3	13	6	69
I-MISC	2	27	7	2	7	145	2	4	150
I-ORG	36	11	47	4	38	5	263	5	342
I-PER	6	2	5	102	0	0	1	292	899
O	3	10	4	0	1	10	11	1	42719

P: 0.8114098816314947 R: 0.5475891713668707 F1: 0.6374572803907079

Figure 17: Logistic Regression Classifier with Token only

length of the token was immediately removed from the list of possible features and was not included in the feature ablation stage.

LinearSVC

LinearSVC had the best performance of all classifiers (Figure 20). It did very well in all categories, with the exception of I-MISC, which had a recall of 0.679, but a very good precision of 0.848. Apart from O, which scored highly on all classifiers, the highest scoring label was I-PER with a precision of 0.895 and a recall of 0.956. Although hyperparameter tuning was performed in this classifier, the best values turned out to be the default values, and the results stayed the same.

LinearSVC combined with word embeddings and traditional one-hot features had promising results (Figure 21). Since word embeddings work best with non-linear classifiers, the results are quite high and could be even higher if an option to use Gaussian kernels was feasible. All in all, an F1-score of 0.855 is quite high, and the scores of precision and recall are balanced. I-MISC is the only class that has somewhat lower scores, but that can also be explained by its underrepresentation in both the test and the development sets.

Overall, the classifiers improved with the use of both traditional one-hot features and word embeddings, highlighting the importance of feature engineering and the understanding that not all features work well with all types of classifiers.

	precision	recall	f1-score	support
B-LOC	0.869	0.818	0.843	1837
B-MISC	0.893	0.782	0.834	922
B-ORG	0.805	0.738	0.770	1341
B-PER	0.864	0.893	0.878	1842
I-LOC	0.907	0.720	0.803	257
I-MISC	0.898	0.633	0.742	346
I-ORG	0.825	0.711	0.764	751
I-PER	0.848	0.937	0.891	1307
O	0.988	0.997	0.992	42759
accuracy			0.967	51362
macro avg	0.877	0.803	0.835	51362
weighted avg	0.966	0.967	0.966	51362

Figure 18: Logistic Regression with all features

	precision	recall	f1-score	support
B-LOC	0.666	0.911	0.770	1837
B-MISC	0.901	0.670	0.769	922
B-ORG	0.733	0.701	0.717	1341
B-PER	0.891	0.801	0.844	1842
I-LOC	0.879	0.198	0.324	257
I-MISC	0.976	0.231	0.374	346
I-ORG	0.717	0.578	0.640	751
I-PER	0.871	0.871	0.871	1307
O	0.980	0.990	0.985	42759
accuracy			0.949	51362
macro avg	0.846	0.661	0.699	51362
weighted avg	0.951	0.949	0.946	51362

Figure 19: Naive Bayes with all features

	precision	recall	f1-score	support
B-LOC	0.910	0.868	0.889	1837
B-MISC	0.895	0.822	0.857	922
B-ORG	0.847	0.807	0.826	1341
B-PER	0.900	0.915	0.907	1842
I-LOC	0.893	0.809	0.849	257
I-MISC	0.848	0.679	0.754	346
I-ORG	0.859	0.739	0.795	751
I-PER	0.895	0.956	0.925	1307
O	0.990	0.997	0.994	42759
accuracy			0.974	51362
macro avg	0.893	0.844	0.866	51362
weighted avg	0.973	0.974	0.973	51362

Figure 20: LinearSVC with all features

	precision	recall	f1-score	support
B-LOC	0.888	0.895	0.892	1837
B-MISC	0.866	0.824	0.844	922
B-ORG	0.833	0.802	0.817	1341
B-PER	0.909	0.933	0.921	1842
I-LOC	0.843	0.794	0.818	257
I-MISC	0.776	0.630	0.695	346
I-ORG	0.832	0.706	0.764	751
I-PER	0.961	0.945	0.953	1307
O	0.991	0.996	0.993	42759
accuracy			0.973	51362
macro avg	0.878	0.836	0.855	51362
weighted avg	0.972	0.973	0.973	51362

Figure 21: LinearSVC with embeddings and one-hot features